Computer Organization and Architecture (CSL251)
LAB-3
2025-26 W
RISC-V Assembly Debugging Using Spike

Function Call

Program: Function Call (Without parameter)

nano fun1.s

```
[~/Desktop/RISCV/program]
umesh > cat fun1.s
.section .text
.globl main

main:
    call add        # call function (no parameters)
    ret             # return from main


add:
    li t0, 5        # first number (inside function)
    li t1, 3        # second number (inside function)
    add t0, t0, t1  # t0 = 5 + 3
    ret
```

Compile the Program(produces the executable file fun1.elf):

riscv64-unknown-elf-gcc fun1.s -o fun1.elf

Disassemble the Executable:

riscv64-unknown-elf-objdump -d fun1.elf > fun1.dump

```
[~/Desktop/RISCV/program]
umesh > riscv64-unknown-elf-gcc fun1.s -o fun1.elf

[~/Desktop/RISCV/program]
umesh > riscv64-unknown-elf-objdump -d fun1.elf > fun1.dump
```

Locate the <main> label and Copy the address of the first instruction:

vi fun1.dump

Exit vi editor

Start Spike Debugger:

spike -d pk fun1.elf

```
[~/Desktop/RISCV/program]
umesh  spike -d pk fun1.elf
```

Initial register dump (before executing code):

(spike) reg 0

```
(spike) reg 0
zero: 0x0000000000000000  ra: 0x0000000000010192  sp: 0x0000003fffffb50  gp: 0x0000000000012818
  tp: 0x0000000000000000  t0: 0x00000000000107ae  t1: 0x000000000000000f  t2: 0x0000000000000000
  s0: 0x0000000000000000  s1: 0x0000000000000000  a0: 0x0000000000000001  a1: 0x0000003fffffb58
  a2: 0x0000000000000000  a3: 0x0000000000000001  a4: 0x0000000000000000  a5: 0x0000000000000000
  a6: 0x000000000000001f  a7: 0x0000000000000000  s2: 0x0000000000000000  s3: 0x0000000000000000
  s4: 0x0000000000000000  s5: 0x0000000000000000  s6: 0x0000000000000000  s7: 0x0000000000000000
  s8: 0x0000000000000000  s9: 0x0000000000000000 s10: 0x0000000000000000 s11: 0x0000000000000000
  t3: 0x0000000000000000  t4: 0x0000000000000000  t5: 0x0000000000000000  t6: 0x0000000000000000
```

Run Until main (Execute instructions until PC reaches 0x101dc, stop just before executing it)

(spike) until pc 0 101dc

```
[~/Desktop/RISCV/program]
umesh  spike -d pk fun1.elf
(spike) until pc 0 101dc
```

Instruction-by-Instruction Execution

Instruction 1 (just press enter that shows):

```
(spike)
core   0: 0x00000000000101dc (0x006000ef) jal     pc + 0x6
(spike)
core   0: 0x00000000000101e2 (0x00004295) c.li    t0, 5
(spike)
core   0: 0x00000000000101e4 (0x0000430d) c.li    t1, 3
(spike)
core   0: 0x00000000000101e6 (0x0000929a) c.add   t0, t1
(spike)
core   0: 0x00000000000101e8 (0x00008082) ret
(spike)
core   0: 0x00000000000101e0 (0x00008082) ret
(spike) reg 0
zero: 0x0000000000000000  ra: 0x00000000000101e0  sp: 0x0000003ffffffb50  gp: 0x0000000000012818
  tp: 0x0000000000000000  t0: 0x0000000000000008  t1: 0x0000000000000003  t2: 0x0000000000000000
  s0: 0x0000000000000000  s1: 0x0000000000000000  a0: 0x0000000000000001  a1: 0x0000003ffffffb58
  a2: 0x0000000000000000  a3: 0x0000000000000001  a4: 0x0000000000000000  a5: 0x0000000000000000
  a6: 0x000000000000001f  a7: 0x0000000000000000  s2: 0x0000000000000000  s3: 0x0000000000000000
  s4: 0x0000000000000000  s5: 0x0000000000000000  s6: 0x0000000000000000  s7: 0x0000000000000000
  s8: 0x0000000000000000  s9: 0x0000000000000000 s10: 0x0000000000000000 s11: 0x0000000000000000
  t3: 0x0000000000000000  t4: 0x0000000000000000  t5: 0x0000000000000000  t6: 0x0000000000000000
(spike) q
```

call add # jal pc + 0x6 // call the function add

call is a pseudo-instruction.

It expands to:

- **auipc ra, %pcrel_hi(add) //upper 20 bits of the PC-relative offset to add**
- **jalr ra, %pcrel_lo(add)(ra) //adds the low 12 bits to ra , jump, stored return address in ra**

Instruction 2 (just press enter that shows):

Loads constant 5 into register t0 (temporary register)

Instruction 3 (just press enter that shows):

t0 = t0 + t1 // register to register arithmetic operation

Instruction 5(just press enter that shows):

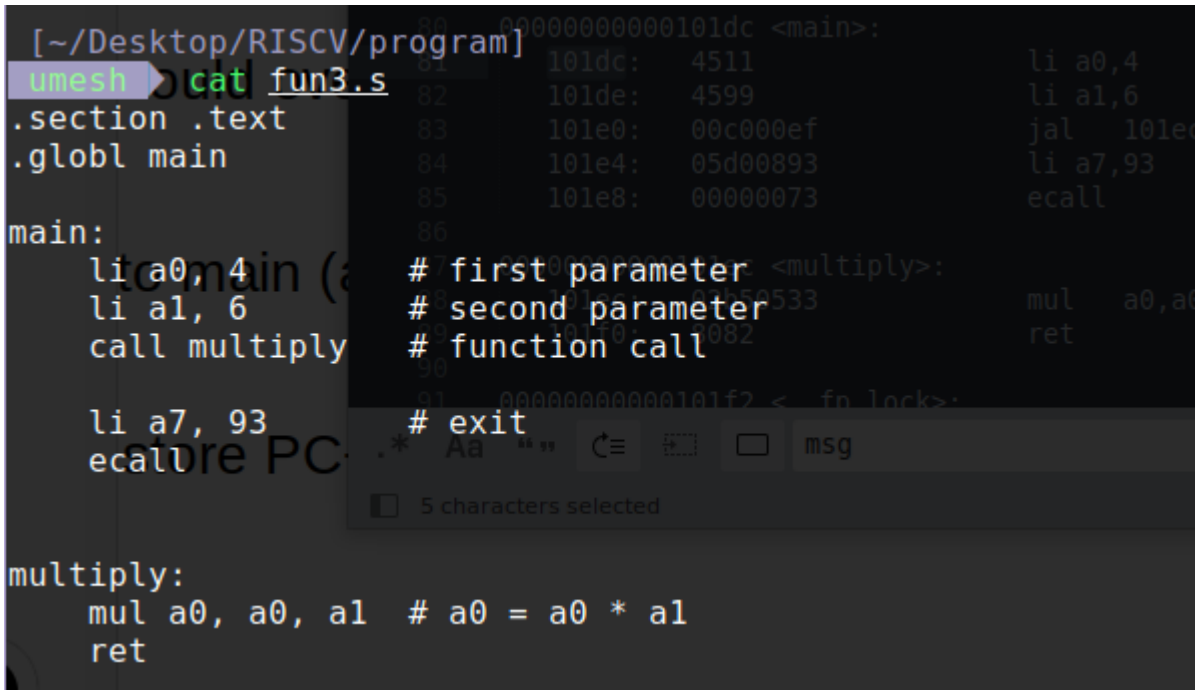
ret

jalr x0, 0(ra)

- **Jumps back to the instruction after call add.**
- **Uses the return address stored in ra**

Program: Function Call (Without parameter)

nano fun3.s



```
[~/Desktop/RISCV/program]
umesh > cat fun3.s
.section .text
.globl main

main:
    li a0, 4          # first parameter
    li a1, 6          # second parameter
    call multiply     # function call

    li a7, 93         # exit
    ecall


multiply:
    mul a0, a0, a1  # a0 = a0 * a1
    ret
```

Compile the Program(produces the executable file fun3.elf):

```
riscv64-unknown-elf-gcc fun3.s -o fun3.elf
```

Disassemble the Executable:

```
riscv64-unknown-elf-objdump -d fun3.elf > fun3.dump
```

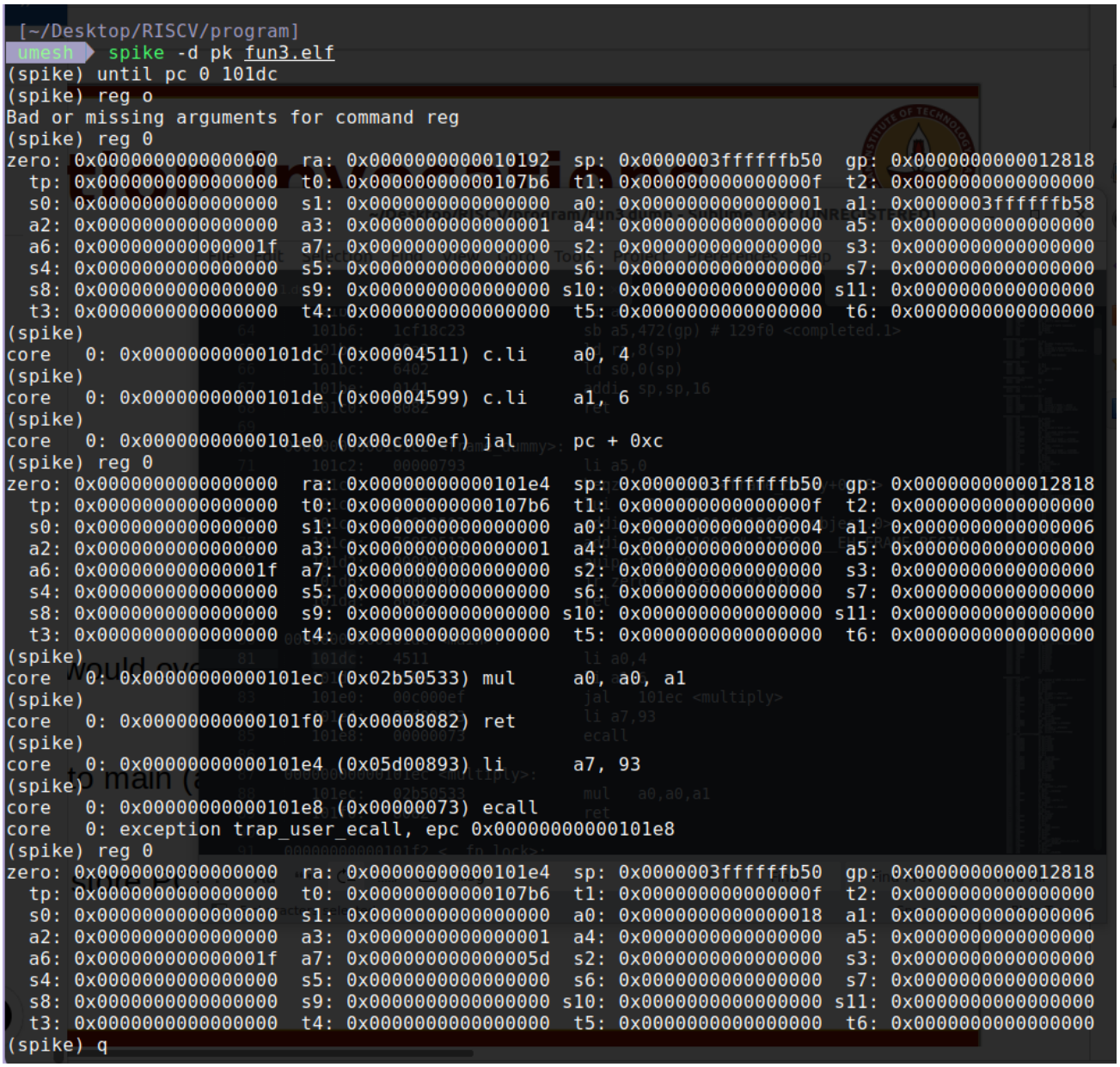
Locate the <main> label and Copy the address of the first instruction:

```
vi fun1.dump
```

Exit vi editor

Start Spike Debugger:

```
spike -d pk fun1.elf
```



Run Until main (Execute instructions until PC reaches 0x101dc, stop just before executing it)

(spike) until pc 0 101dc

Initial register dump (before executing code):

(spike) reg 0

Instruction-by-Instruction Execution

Instruction 1 (justpress enter that shows):

li a0 ,4 // first parameter passing

Instruction 2 (justpress enter that shows):

li a1 ,6 // second parameter passing

Instruction 3 (justpress enter that shows):

jal pc + 0xc // fuction call

Instruction 4 (justpress enter that shows):

mul a0 , a0, a1 // the values of a0 =4 and a1 =6

Instruction 5 (justpress enter that shows):

ret // return address

Instruction 6 (justpress enter that shows):

li a7, 93 // exit system call

a7 holds the system call number

93 = exit syscall in Linux RISC-V . Return value in a0 becomes exit status

Instruction 7 (justpress enter that shows):

ecall

Triggers a trap to the OS kernel

Kernel checks: //a7 → syscall number //a0 → exit code Program terminates